Rung 0: RPS Regret Matching — Post-PR Writeup

drawmaha-solver PR #1: the learning rule, proven on the smallest game

What actually shipped, and how

Companion artifact: rung0-rps-writeup.md. Synthesized by Claude from the merged diff and local test runs.

1 Outcome & decision summary

The drawmaha-solver project is building a neural poker solver for heads-up Drawmaha (a split-pot draw/Omaha hybrid with no existing solver), developed as a ladder of five rungs: each rung adds one layer of complexity and is verified against a known answer before climbing. This PR ships **rung 0** — the *regret-matching* learning rule, the algorithm at the heart of every later rung, implemented on rock-paper-scissors, the smallest game with a known mixed-strategy equilibrium. It also establishes the Python package, test harness, and figure pipeline every later rung builds on.

Outcome

The learner provably works — its average strategy converges to the known Nash equilibrium (a best-responding adversary wins only 0.0009 chips/round off it after 100k iterations), and against a biased opponent it converges to the exploiting counter. **The ask: approve and merge PR #1 into** main**.** No migrations, no deploys, no ordering conditions. All 38 tests pass locally; the repository has no CI yet (the one gray tile in §4; follow-up in §6). Verdict: **merge-ready**.

Table 1. The PR, in plain words.

PR	What it does	Size
#1	Adds the rung-0 package: the RPS rules, the regret-matching learner, a match runner with human/fixed/learner players, a convergence analysis that certifies the learner finds the Nash equilibrium (four committed figures), and a terminal game to play against it.	18 files, +1,310

2 The algorithm, and the as-built map

Everything in this PR lives in one Python package plus its tests; the map below shows the two ways in and what they share. Four terms carry the whole document. **Regret**: after a round, for each action you *could* have played, how much better it would have scored than what you did. **Regret matching**: the learning rule — next round, play each action with probability proportional to its accumulated *positive* regret (uniform if none is positive). **Current vs. average strategy**: the current strategy cycles forever and never settles; the theorem is that the *running average* of all strategies played converges to the Nash equilibrium — the average is the product. **Exploitability**:

the score — how much a best-responding adversary who knows your strategy wins per round against it; exactly 0 at the Nash equilibrium, and the project's evaluation currency all the way up the ladder.

Example

The ledger after one round, starting uniform $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$: the opponent shows paper, so the utility of each of our actions is (rock -1, paper 0, scissors $+1$). Our strategy's expected utility is 0, so the regret increments are the utilities themselves: $(-1, 0, +1)$. Only scissors has positive regret, so the next round plays scissors with probability 1 — and the average strategy has banked one uniform round.

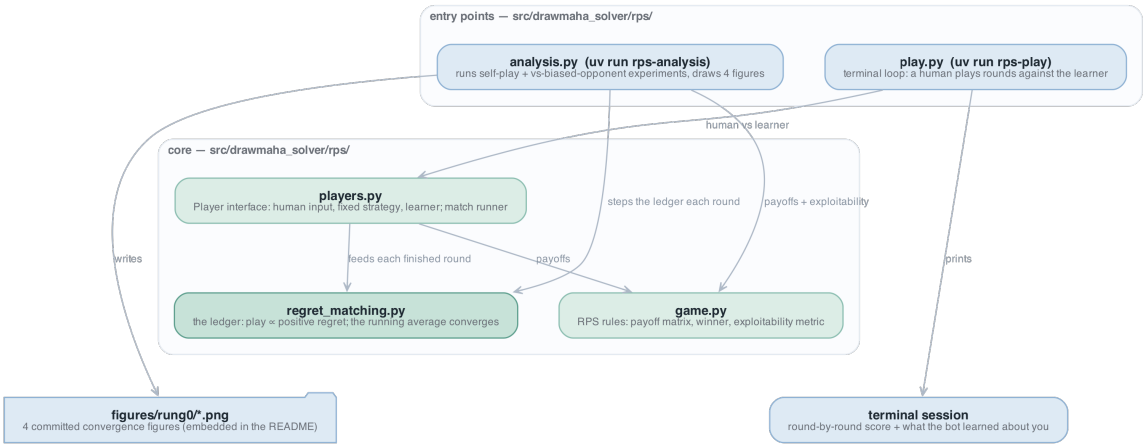

Figure 1. The package as merged. Green = new code (everything is new at rung 0); blue = entry points and outputs. Two entry points branch into a shared core: the analysis CLI drives the ledger directly, the play CLI goes through the player layer, and both converge on the ledger and the rules.

3 How it works

One PR, five modules, walked in the diagram's causal order. Per file: what it is, then the facts that matter.

game.py — the rules of RPS, and the measuring stick.

- the payoff matrix (`PAYOFF[a, b]` = row player's chips) is the single source of truth; zero-sum and antisymmetric by construction [source: src/drawmaha_solver/rps/game.py:32]
- `winner(a, b)` derives win/loss/tie from the matrix, never from a second rules table [source: game.py:44]
- `best_response_value(strategy)` is the exploitability metric — the project's first evaluation instrument [source: game.py:61]
- actions are an `IntEnum` so they index payoff matrices and strategy vectors directly [source: game.py:21]

regret_matching.py — the learner: one ledger, ~40 lines, the heart of the project.

- `strategy()` normalizes positive accumulated regrets; all-negative regrets fall back to uniform [source: regret_matching.py:36]
- `update(utilities)` accumulates regret against the strategy's *expected* utility and banks the strategy into the running average [source: regret_matching.py:51]
- `average_strategy()` returns the running average — the object that converges [source: regret_matching.py:44]
- a one-action ledger is rejected loudly rather than degenerating silently [source: regret_matching.py:31]

```
def update(self, utilities: np.ndarray) -> None:
    current = self.strategy()
    self.strategy_sum += current
    self.cumulative_regret += utilities - current @ utilities
```

Decision

The ledger measures regret against the strategy's expected utility $u - \langle \sigma, u \rangle$, not against the sampled action (the textbook RPS-trainer variant) — *because* that is exactly the counterfactual-regret form CFR uses at every information set, so rung 1 changes nothing about the update rule; and it is lower-variance: final exploitability 0.00091 chips/round vs 0.00443 for the sampled form on the same budget. Don't "simplify" it back.

players.py — the glue: who plays, and how rounds flow.

- `Player` is a two-method interface: `act(rng)` and `observe(own, opp)` [source: players.py:19]
- `RegretMatchingPlayer.observe` feeds the ledger `PAYOFF[:, opp]` — the utility of every action against the opponent's revealed move [source: players.py:53]
- `FixedStrategyPlayer` rejects any input that is not a probability distribution [source: players.py:37]
- `play_match(p0, p1, *, n_rounds, rng)` runs rounds, shows both players the result, returns player 0's payoffs [source: players.py:85]

analysis.py — the certification: experiments plus the four committed figures.

- `run_self_play` records, per iteration, the current strategy, the running average, and the average's exploitability [source: analysis.py:98]
- `run_vs_fixed` pits the learner against a fixed 50%-rock opponent [source: analysis.py:121]
- results travel as frozen dataclasses, not loose dicts [source: analysis.py:49]
- four figures (average-strategy convergence, current-vs-average cycling, log–log exploitability decay, best-response learning) write to figures/rung0/ [source: analysis.py:139]

play.py — the demo: `uv run rps-play` is a terminal loop against the learner; on quit it prints the bot's learned average strategy — against a habit-prone human it visibly drifts toward the counter [source: play.py:10].

4 Test results

All 38 tests pass, run locally on this branch (`uv run pytest`, 1.4 s). Every count below is paired with what it proves; the one gray tile is the *absence* of CI, not a failing check.

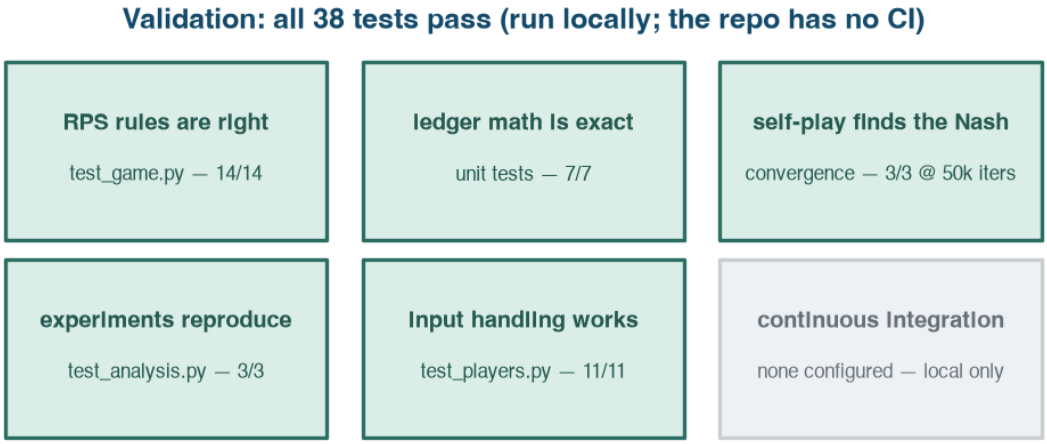

Figure 2. Teal = passed locally; gray = not verifiable (the repository has no CI). Tile titles say what passing proves.

- **14 rules tests** prove the game itself is right: all 9 action-pair outcomes pinned individually, the payoff matrix verified zero-sum, and the exploitability metric anchored at both extremes (uniform → 0, pure rock → 1) plus a hand-computed midpoint.
- **7 ledger unit tests** prove the math is exact, not just plausible: hand-computed regret increments for the expected-utility update, the positive-regret normalization, the uniform fallback, and the average-strategy arithmetic.
- **3 convergence tests** prove the theorem shows up in practice: 50k-iteration self-play lands within 0.02 of the uniform Nash with exploitability < 0.02 for *both* players; against a 50%-rock opponent the average goes $> 90\%$ paper and realized winnings beat $+0.1$/round; identical seeds reproduce identical matches.
- **3 analysis tests** prove the certification pipeline reproduces: trajectory shapes, exploitability falling over time, and all four figure files actually written.
- **11 input-handling tests** prove the boundary is defended: non-distributions rejected, `r/p/s` and full words parsed, garbage reprompted, quit raised cleanly.

Headline numbers from the committed 100k-iteration run: self-play average strategy $(0.334, 0.333, 0.333)$, exploitable for **0.00091 chips/round**; against 50% rock the learner's average is 99.9% paper earning +**0.237/round** (the theoretical best response earns +0.25).

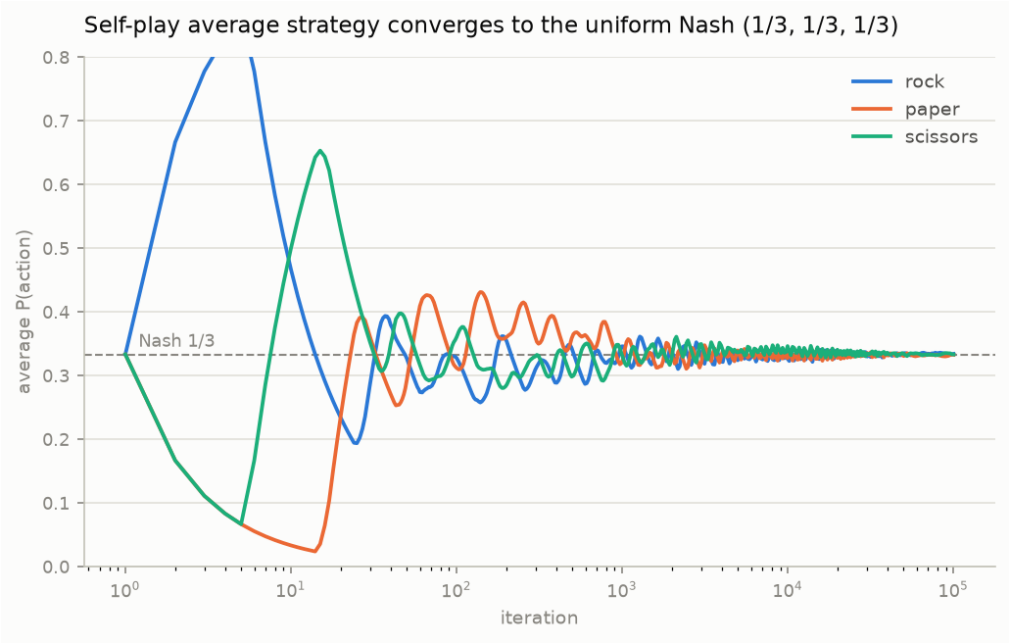

Figure 3. What convergence actually looks like: each action's average probability (log-scale iterations) swinging early, then locking onto the $1/3$ line. The other three committed figures show the current strategy cycling forever, the $1/\sqrt{T}$ exploitability decay, and the best-response convergence.

5 Edge cases & handling

The boundaries the change has to survive, each with its current status and where it is handled.

Table 2. Edge cases & handling. Paths are under src/drawmaha_solver/rps/.

Case	Status	Where handled
One-action ledger (degenerate game)	handled — raises	regret_matching.py:31
All regrets negative (no positive signal)	handled — uniform	regret_matching.py:40
Player given a non-distribution strategy	handled — raises	players.py:37
Garbage terminal input	handled — re-prompts	players.py:79
Quit before playing any round	handled — no summary	play.py:38
Analysis run shorter than the 5,000-iteration plot window	handled — clamps	analysis.py:156
`update()` given a wrong-length utilities vector	*known gap*	see below

The known gap: numpy raises on the length mismatch, but only *after* the strategy was banked into the average — the ledger is left half-updated [source: regret_matching.py:51]. Accepted at

rung 0 (the only callers are in-repo and tested); harden the boundary when the API grows callers at rung 1.

6 Risks & follow-ups

Only what is still true after merge.

Risk

Nothing guards `main`**:** the repository has no CI, so these 38 tests run only when someone runs them — a future PR could break the ladder's foundation silently → add a GitHub Actions `uv run pytest` workflow before rung 1 lands.

- **Convergence tests are seed-pinned** — a numpy RNG behavior change could flip a threshold → `uv.lock` is committed and pins numpy; keep it that way.
- **README numbers and figures regenerate only manually** — if the ledger changes, `uv run rps-analysis` must be re-run or the committed figures drift from the code.
- **The interactive play loop has no test of its own** — input *parsing* is tested (11 tests); the `play.main()` round loop is exercised only by hand.
- **Deferred (by design):** rung 1 — tabular CFR on Kuhn poker against the known exact equilibrium — is next; nothing in this PR blocks its shape beyond the update rule deliberately matching CFR's.

7 Validation & merge-readiness gate

The final verdict, echoing §1's ask.

Table 3. Merge-readiness gate.

Gate	Status	Note
Branch / commits	`alec/rung0-rps`	983066b, 623b96b — PR #1
Merge conflicts	none	`mergeable`: MERGEABLE, state CLEAN
CI / checks	n/a	no CI configured on this repository (§6)
Tests (local)	38/38 pass	`uv run pytest`, verified on the PR tip
Review	0 open	no human review yet; single-author repository
Migrations / deploy	none	pure library + CLI code

Bottom line: safe to merge as-is; the only outstanding items are the no-CI follow-up and the one known gap, both accepted for rung 0.